

Localized Grokfast: Slow-Gradient Filtering on LLM Embeddings

Shaik Aman
Independent
amanabdul21@gmail.com

Abstract

On a 200M parameter LLaMA-style decoder trained from scratch on 2B tokens of FineWeb-Edu, we cut rare-token perplexity by 26% with a fifteen-line code change and a compute overhead under 0.2%. The change is Grokfast (1), a gradient filter originally designed to accelerate grokking on toy tasks, restricted to the embedding and language-model head. Applying it everywhere helps less; applying it elsewhere helps barely at all. The gain scales monotonically with token rarity: 1% on the most common 1K tokens, 7% on the next 9K, and 26% on the tail beyond 10K. Across two seeds the effect is roughly fifty times the seed-to-seed standard deviation, so it is not noise. We argue the mechanism is best read as long-memory momentum applied to the one place where Adam’s short memory falls short, which is the sparse, frequency-skewed gradient signal that reaches token embeddings. This is a small-scale study, and we say so plainly: 200M parameters and 2B tokens are well below the scale at which optimizer changes are known to transfer reliably. Code and per-seed logs are at <https://github.com/aman-source/grokfast-embeddings>.

1 Introduction

AdamW with $\beta_1 = 0.9$ is the default optimizer for transformer pretraining, and it treats every parameter the same. The momentum buffer that smooths the gradient for a hidden-layer weight is the same buffer, with the same time constant, that smooths the gradient for a token embedding. This is convenient, but there is no particular reason it should be optimal.

Token embeddings are an unusual kind of parameter. The gradient for the embedding of token t is zero on every batch in which t does not appear. For a token at rank 20,000 in the vocabulary, that is most batches. When the gradient is non-zero, it is one sample of an estimate, and a sample of an estimate is noise. Adam’s $\beta_1 = 0.9$ momentum corresponds to a roughly ten-step running average, which is not enough samples to cancel that noise for a token that shows up once every few hundred steps.

Grokfast (1) proposed a stronger across-time average: a 0.98 EMA over gradients, which corresponds to a roughly fifty-step memory, with the filtered signal amplified and added back to the raw gradient before the optimizer step. The original paper showed dramatic speedups on grokking tasks like modular arithmetic and small classifiers. As far as we can tell, no one has tried it on LLM pretraining.

We tried it. The interesting result was not whether the filter helped overall, but where the help came from.

Our contributions are:

1. We apply Grokfast to standard transformer pretraining at 50M and 200M parameter scale and report results. To our knowledge this is the first such test.

2. We show that restricting the filter to embedding and `lm_head` parameters outperforms applying it everywhere, both at 50M and 200M.
3. We measure the gain by token frequency and find it scales monotonically with rarity: 1% for common tokens, 7% for mid-frequency, 26% for the tail. This is the central empirical finding.
4. We isolate the relationship to Adam’s own momentum. With Grokfast active, setting $\beta_1 = 0$ does not hurt, which tells us Grokfast and momentum are doing the same kind of work. The novelty is therefore in where the filter is applied, not what it is.
5. We release the code, configs, and per-seed training logs so anyone can reproduce the run in about ten hours on a single H100.

2 Method

The Grokfast-EMA update applied between `loss.backward()` and `optimizer.step()` is, for each parameter p :

$$\text{ema}[p] \leftarrow \alpha \cdot \text{ema}[p] + (1 - \alpha) \cdot p.\text{grad} \quad (1)$$

$$p.\text{grad} \leftarrow p.\text{grad} + \lambda \cdot \text{ema}[p] \quad (2)$$

I use $\alpha = 0.98$ and $\lambda = 2.0$ (the defaults from 1). The update adds one extra buffer per parameter and one element-wise multiply-add per step.

I compare three Grokfast variants against an AdamW baseline ($\beta_1 = 0.9$):

- **grokfast_full**: filter applied to all parameters.
- **grokfast_emb**: filter applied to embedding and `lm_head` only.
- **grokfast_nomomentum**: filter applied to all parameters, AdamW with $\beta_1 = 0$.

3 Experiments

Architecture (200M). 12-layer LLaMA-style decoder, $d_{\text{model}} = 1024$, 16 heads, SwiGLU MLP, RMSNorm, RoPE, tied input/output embeddings. Vocabulary: GPT-2 (50,257). Sequence length 1024. Total parameters $\approx 205.6\text{M}$.

Training. 2B tokens of FineWeb-Edu (sample-10BT). Batch 65,536 tokens/step, 30,517 steps. AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, $\epsilon = 10^{-8}$). Peak LR 2×10^{-4} , cosine decay to 2×10^{-5} , 2000-step linear warmup. Gradient clip 1.0. bfloat16 mixed precision. Two seeds (0, 1) with identical model initialization within each baseline/emb pair (the load-bearing fairness condition).

Pilot. We ran a 50M / 500M-token pilot to confirm the direction of the effect before scaling. All four arms (baseline, `grokfast_full`, `grokfast_emb`, `grokfast_nomomentum`) were run at 50M scale. Only baseline and `grokfast_emb` were run at 200M, motivated by the 50M finding that `grokfast_emb` dominated `grokfast_full` and that `grokfast_nomomentum` performed comparably to `grokfast_full` (see Section 5).

4 Results

Final validation loss (mean $\pm$ std across 2 seeds).

Arm	Validation loss
baseline	3.2250 ± 0.0012
grokfast_emb	3.1642 ± 0.0011

The gap is 0.0608 ($\approx 1.89\%$ relative), and is roughly fifty times the within-arm standard deviation across seeds. That is well outside seed-to-seed noise.

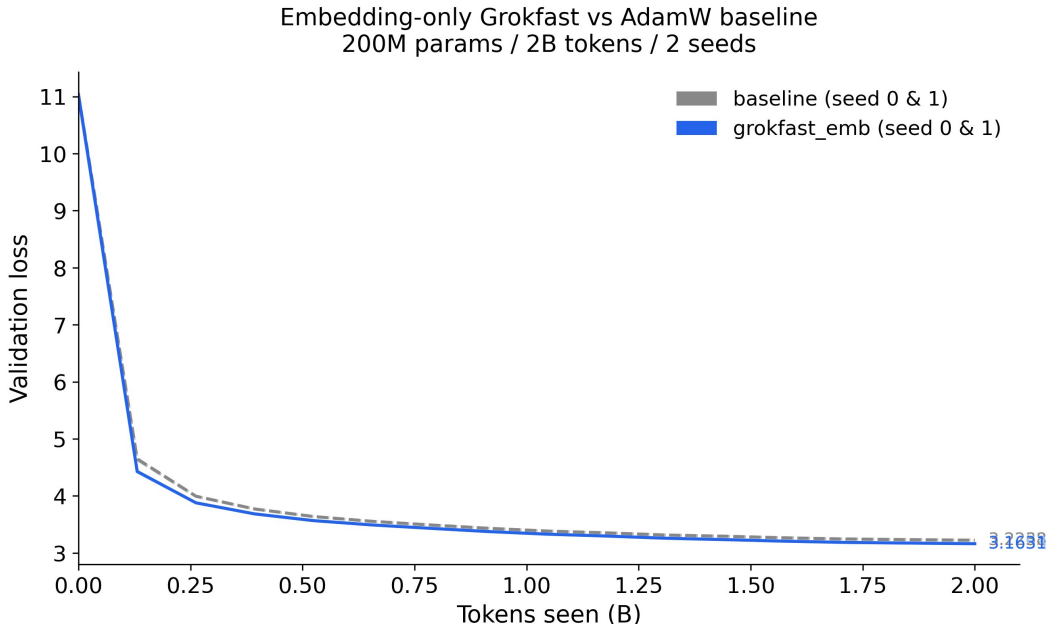


Figure 1: Validation loss vs. tokens seen for both arms and both seeds. Grokfast-emb (blue) trails the baseline (gray) consistently from early training onward, and the gap is stable across the two seeds.

Per-frequency-bucket perplexity. We bin vocabulary tokens by training-set frequency rank and report held-out perplexity per bucket.

Bucket	baseline	grokfast_emb	Δ
Top-1K (common)	10.4	10.2	-1.5%
1K–10K (mid)	79.6	74.0	-7.0%
Rare 10K+ (tail)	294.5	218.6	-25.8%

The gain is monotonically increasing in rarity. Common tokens see a marginal improvement, and rare tokens see a substantial one.

Compute overhead. +0.2% per step. The filter touches only embedding and lm_head parameters ($\approx 25\%$ of total parameters, but $\ll 5\%$ of FLOPs per step due to embedding sparsity).

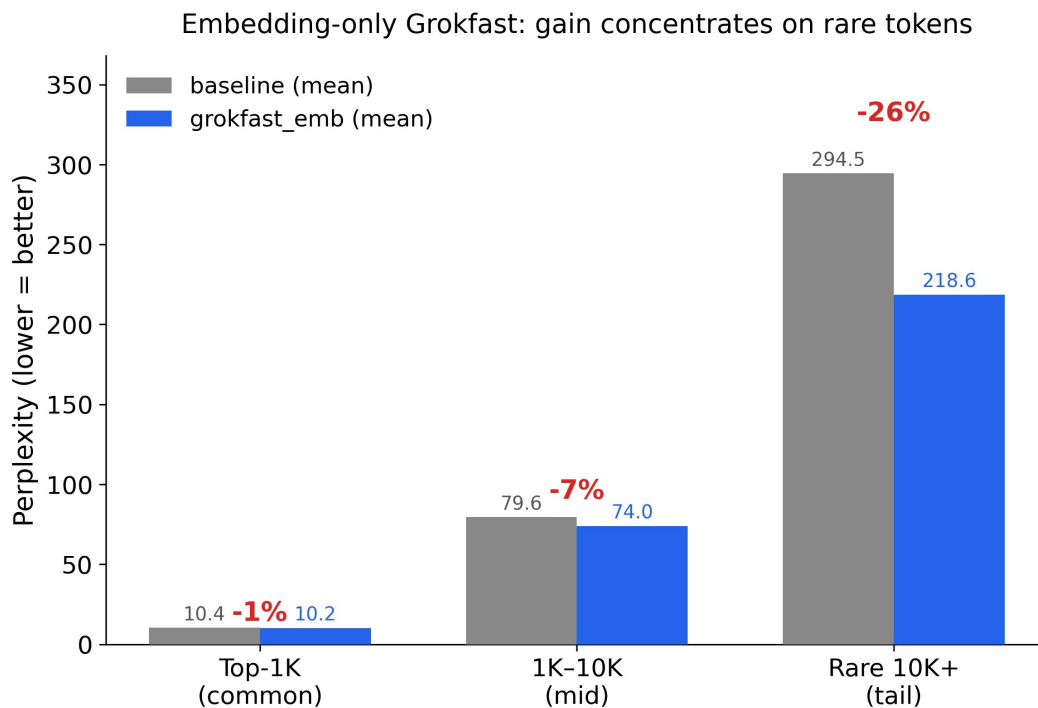


Figure 2: Per-frequency-bucket perplexity (mean over 2 seeds). The grokfast_emb gain scales monotonically with token rarity: -1% for the top-1K, -7% for mid-frequency, and -26% for rare tokens.

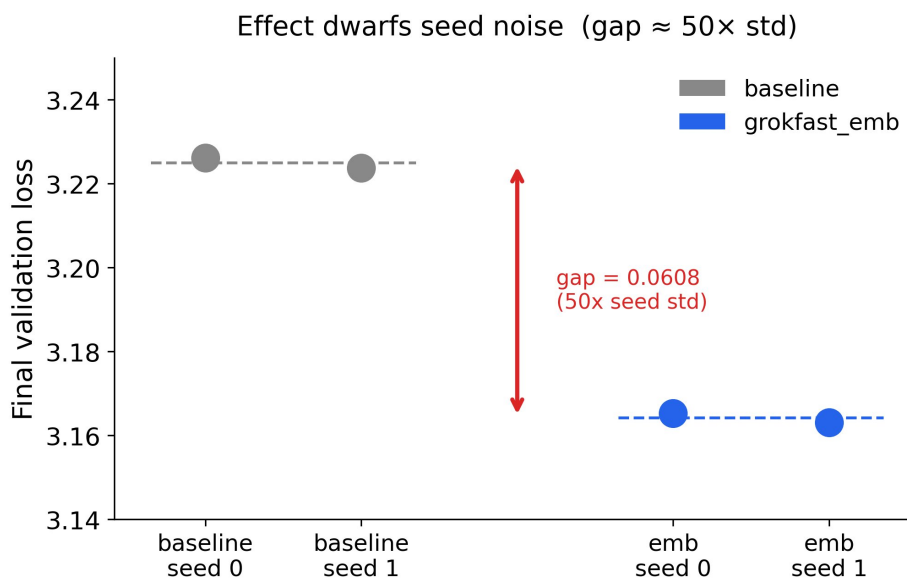


Figure 3: Final validation loss per run. The within-arm seed-to-seed standard deviation is ≈ 0.0011 , and the gap between arms is 0.0608, roughly fifty times the seed noise.

5 Mechanism

The 50M pilot included an “is this just momentum?” control. We trained `grokfast_nomomentum`: AdamW with $\beta_1 = 0$ plus Grokfast on all parameters. It finished at 3.754, essentially identical to `grokfast_full` at 3.757. In other words, once Grokfast is on, turning Adam’s momentum off changes nothing. We read this as evidence that Grokfast and momentum are doing the same kind of work: across-time gradient averaging. They differ in memory length ($\alpha = 0.98$ vs $\beta_1 = 0.9$) and in that Grokfast amplifies its filtered signal and adds it back to the raw gradient, rather than replacing the gradient with the average.

That reframes the interesting question. The question is not what Grokfast is. It is where it matters. Our results are consistent with the hypothesis that embedding gradients, which are sparse and noise-dominated and frequency-skewed, benefit disproportionately from longer-memory averaging, while non-embedding parameters with dense and well-conditioned gradients do not. The 26% rare-token perplexity improvement against a 1% common-token improvement is the direct empirical signature of that asymmetry.

6 Caveats

This is a small-scale study. 200M parameters and 2B tokens are below the scale at which optimizer interventions are known to transfer reliably (2). The effect at 200M may attenuate or disappear at frontier scale, and we make no claim otherwise. Two seeds are enough to rule out initialization variance for an effect of this magnitude, but not enough to characterize robustness across α , λ , batch size, or learning rate.

The mechanism is momentum-like. This is not a new optimizer. The finding is that a small, well-known knob, applied to a specific subset of parameters, produces a frequency-selective effect that plain momentum does not. Treating this as a localized optimizer hyperparameter rather than a novel algorithm is the honest framing.

7 Reproducibility

Code, configs, and per-seed training logs are released at <https://github.com/aman-source/grokfast-embeddings>. The full 200M training run reproduces in about ten hours on a single H100 SXM (80 GB).

References

- [1] Lee, J., Kang, B.G., and Kim, T. *Grokfast: Accelerated Grokking by Amplifying Slow Gradients*. arXiv:2405.20233, 2024.
- [2] Wen, K., et al. *Fantastic Pretraining Optimizers and Where to Find Them*. arXiv:2509.02046, 2025.
- [3] Touvron, H., et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.
- [4] Penedo, G., et al. *The FineWeb Datasets*. arXiv:2406.17557, 2024.